



CloudDrive

Technical Report

Scalable File Storage on AWS — a Mini Google Drive
Final-Year Cloud Computing Project

Author: Shehriyar Ali Rustam

Repository: github.com/Shehriyar-Ali-Rustam/clouddrive

CloudDrive is a cloud-based file-storage web application — a simplified Google Drive — that lets users securely upload, organise, download and share files. It is built to demonstrate core cloud-computing principles: object storage, pre-signed URLs, a managed database, stateless auto-scaling services, infrastructure as code, and CI/CD.

Aspect	Summary
Type	Full-stack web application (REST API + SPA)
Cloud storage	Amazon S3 (live) with pre-signed URLs
Deployment	Render / Docker + AWS ECS (Terraform); live demo via tunnel
Tests	19 automated tests (all passing)
Status	Working, deployed, and version-controlled on GitHub

1. Introduction

1.1 Problem Statement

Storing and sharing files across devices and people is a universal need. Traditional approaches that stream every file through an application server do not scale: the server becomes a bottleneck and a single point of failure. CloudDrive solves this using cloud object storage and pre-signed URLs, keeping the application server stateless and horizontally scalable.

1.2 Objectives

- Provide secure user accounts with authentication and per-user isolation.
- Allow uploading, downloading, organising and deleting files in the cloud.
- Enable sharing — both with registered users and via public expiring links.
- Demonstrate a scalable, cloud-native architecture (S3, RDS, ECS, IAM).
- Automate infrastructure (Terraform) and delivery (CI/CD).

1.3 Scope

The system covers authentication, file/folder management, quotas, sharing, and a polished web UI. It runs in two interchangeable modes — a local mode (SQLite + disk) for offline development and an AWS mode (S3 + PostgreSQL) for the cloud — selected by configuration with no code changes.

2. Key Features

Feature	Description
Authentication	Email + password sign-up/login, JWT tokens, bcrypt-hashed passwords
File upload/download	Browser transfers bytes directly to storage via pre-signed URLs
Organisation	Files and folders, isolated per user
Sharing	Share with a registered user, or create a public expiring link
Storage quota	Per-user limit with a live usage bar
Versioning	S3 bucket versioning keeps previous file versions
Two run modes	Local (SQLite+disk) and Cloud (S3+PostgreSQL) via one setting

3. System Architecture

The design separates concerns cleanly: the database stores only metadata (who owns what, file names, sizes, storage keys, share rules) while Amazon S3 stores the actual file bytes. The browser talks directly to S3 for the heavy data transfer.

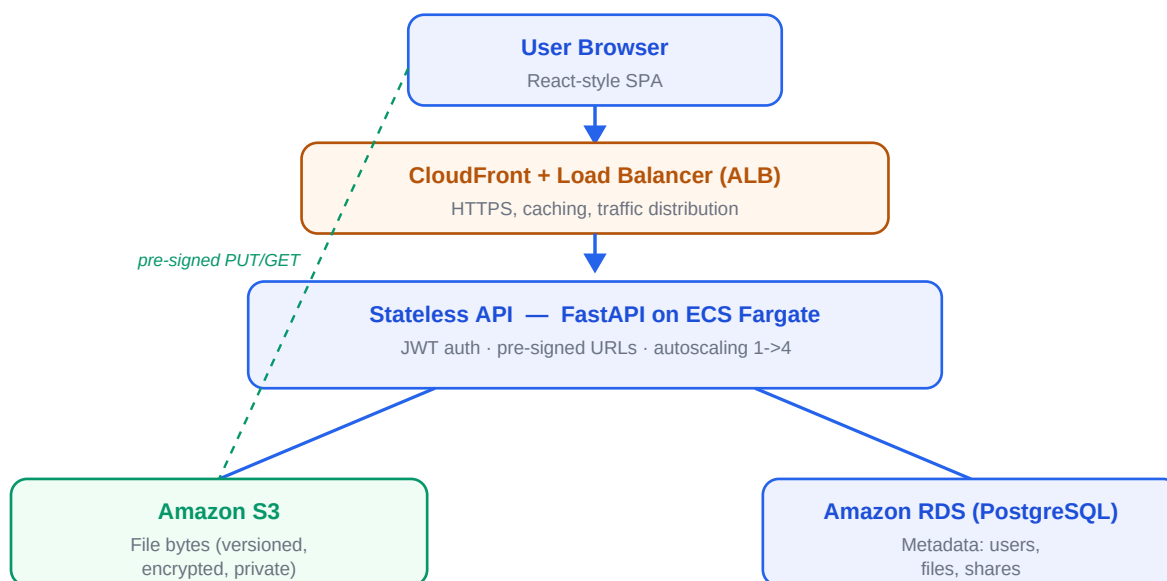


Figure 1: High-level cloud architecture.

3.1 Components

Component	Technology	Responsibility
Frontend	HTML/CSS/JS SPA	Dashboard UI, upload flow, sharing
API	FastAPI on ECS Fargate	Auth, metadata, pre-signed URL issuing
File storage	Amazon S3	Stores file bytes; versioned & encrypted
Database	PostgreSQL (RDS) / SQLite	Users, files, folders, shares
Load balancer	Application Load Balancer	Distributes traffic, health checks
Auth	JWT + bcrypt	Stateless authentication
IaC	Terraform	Provisions the entire AWS stack

4. The Pre-signed URL Upload Flow

This is the central cloud concept. The API never streams file bytes; it only issues a short-lived, signed permission slip (a pre-signed URL) and the browser uploads directly to S3. This keeps the API lightweight and horizontally scalable.

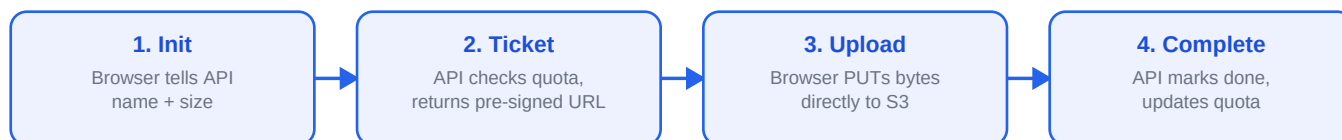


Figure 2: Three-step pre-signed upload sequence.

5. Data Model

Four tables capture all metadata. File bytes live in S3, referenced by a storage key.

Table	Key Fields	Purpose
users	id, email, password_hash, storage_used, storage_quota	Accounts & quota

Table	Key Fields	Purpose
folders	id, name, owner_id, parent_id	Folder hierarchy
files	id, name, s3_key, size, owner_id, folder_id, uploaded	File metadata
shares	id, file_id, shared_with_user_id, public_token, expires_at	Sharing rules

6. Technology Stack

A full illustrated list (with icons) is in [CloudDrive_Tech_Stack.pdf](#). Summary:

Layer	Tools & Services
Language/Backend	Python 3.12, FastAPI, Uvicorn, SQLAlchemy, Pydantic
Frontend	HTML5, CSS3, Vanilla JavaScript
Security	JWT (python-jose), bcrypt
Storage/DB	Amazon S3, PostgreSQL, SQLite, psycopg2, boto3
AWS	S3, IAM, RDS, ECS Fargate, ALB, VPC, ECR
DevOps	Terraform, Docker, GitHub Actions, Git, pytest
Hosting	Render, Cloudflare Tunnel

7. Security Model

Control	How it protects the system
Password hashing	bcrypt one-way hashes; plaintext never stored
JWT auth	Signed, expiring bearer tokens on every API call
Pre-signed URLs	Time-limited (15 min), scoped to a single object
Private bucket	S3 blocks all public access; encrypted at rest (AES-256)
IAM least-privilege	API role may touch only this one bucket
Per-user isolation	Every query filtered by owner_id
Quota enforcement	Prevents a single user exhausting storage

8. User Interface

The following are real screenshots of the running application (storage badge shows S3).

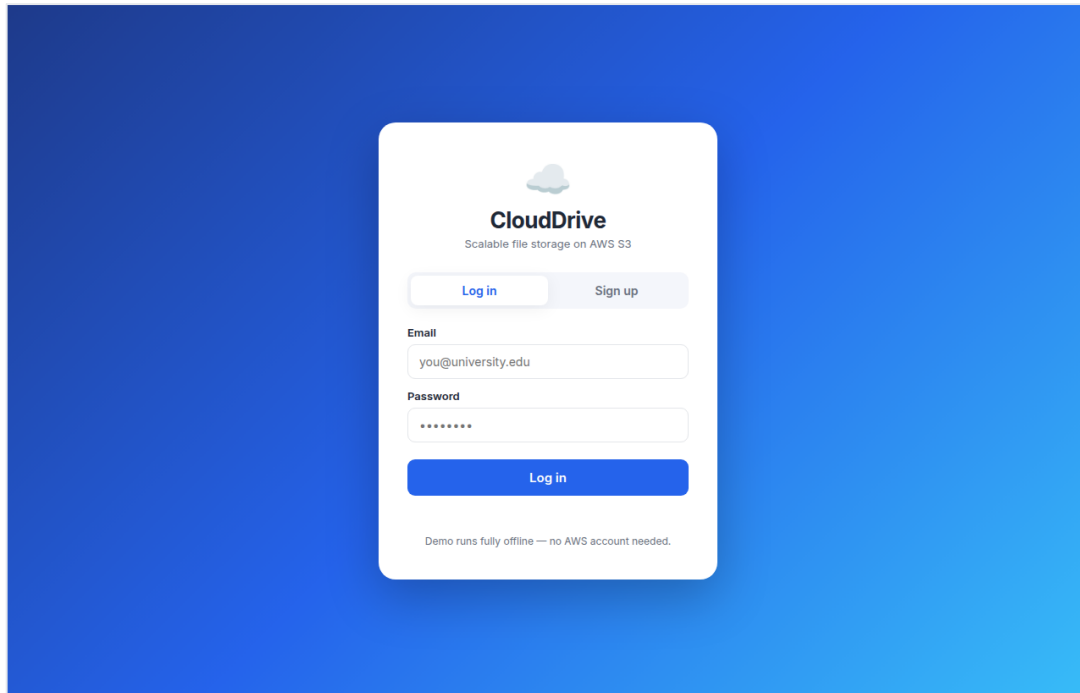


Figure 3: Sign-up / login screen.

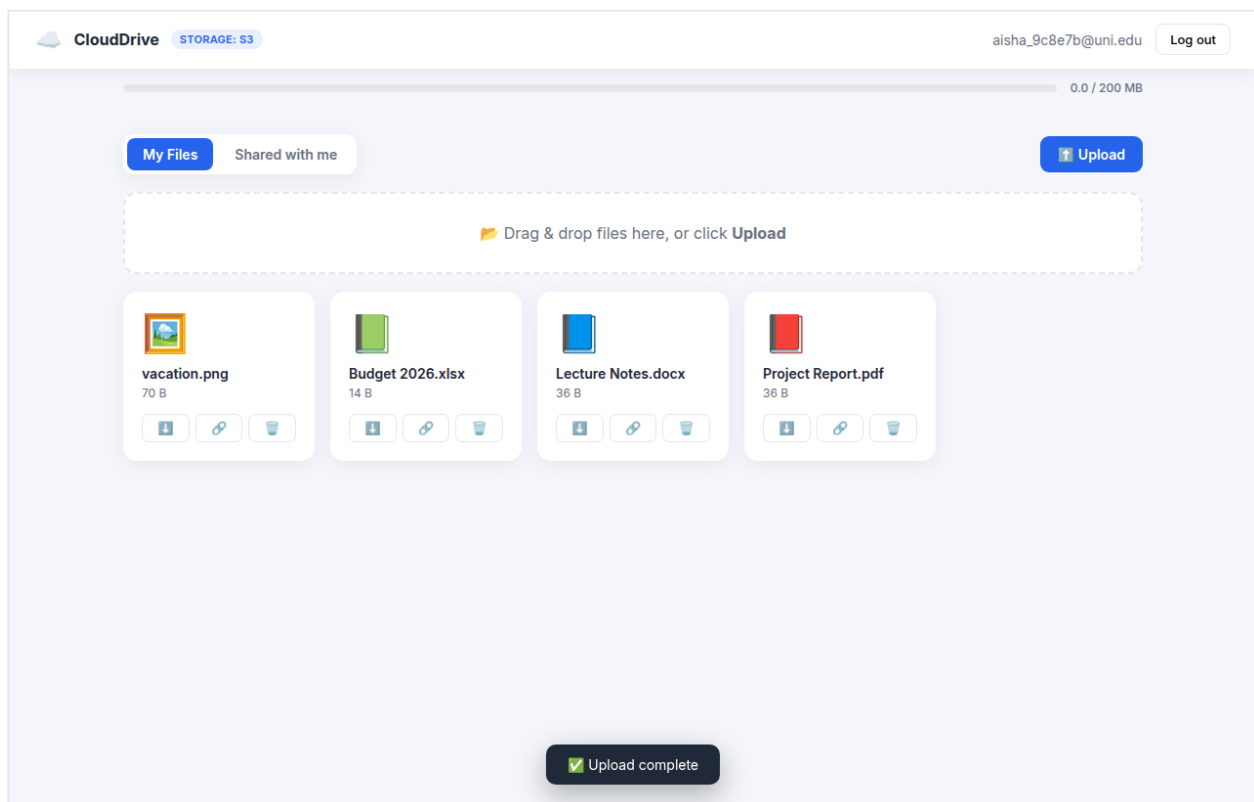


Figure 4: Dashboard — files on AWS S3, quota bar, type icons.

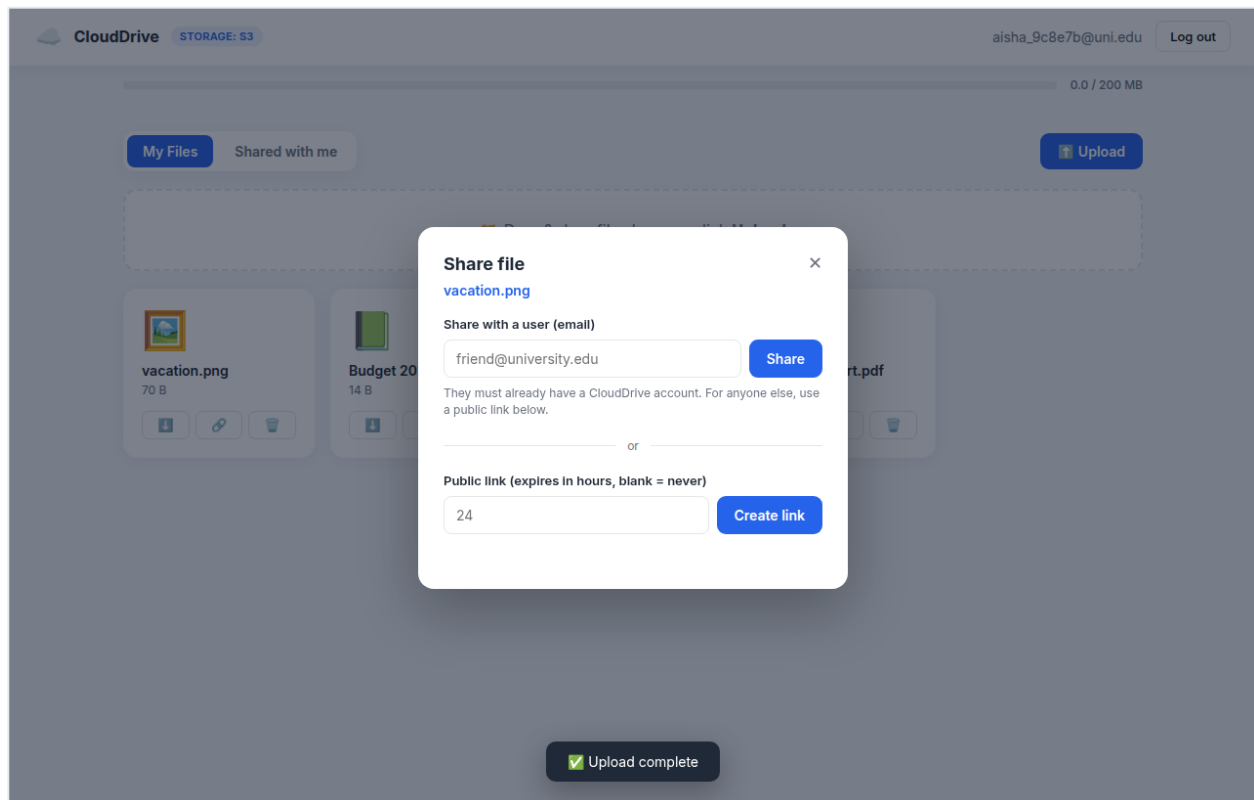


Figure 5: Share dialog — share with a user or create a public link.

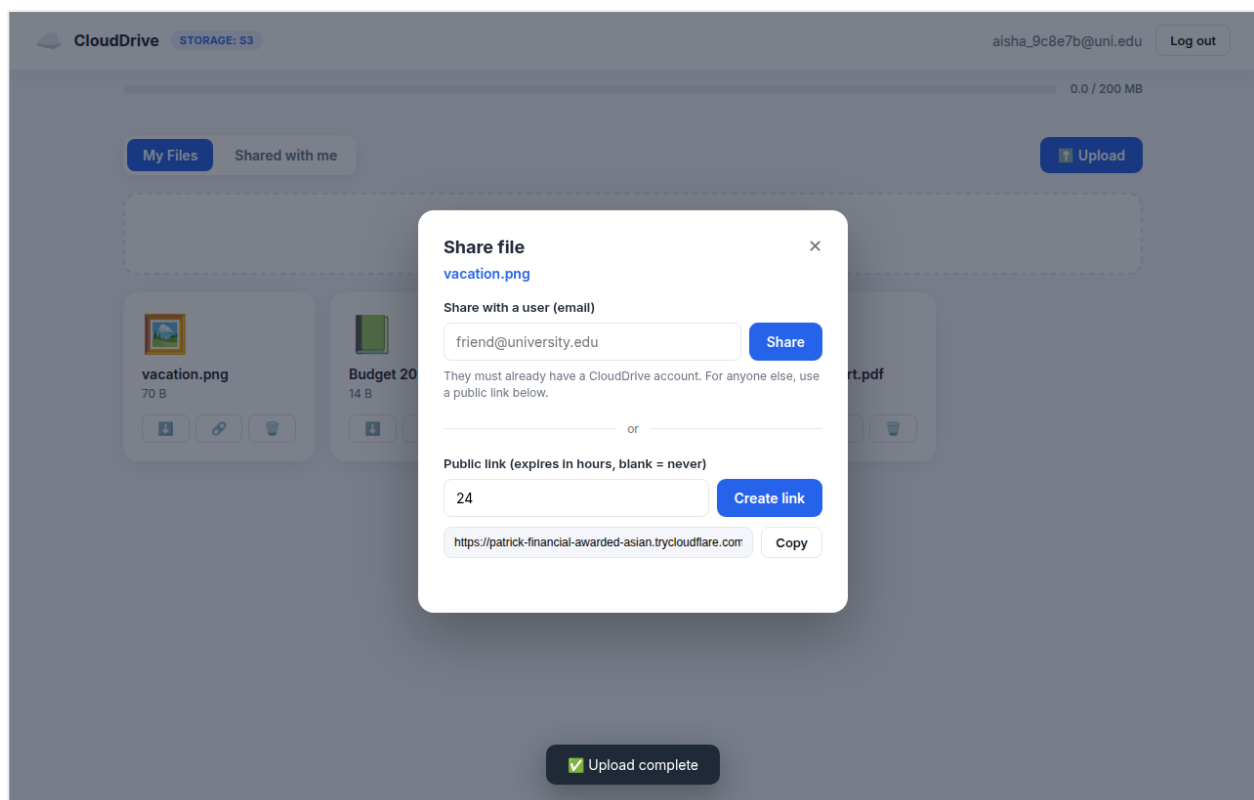


Figure 6: A generated public expiring link.

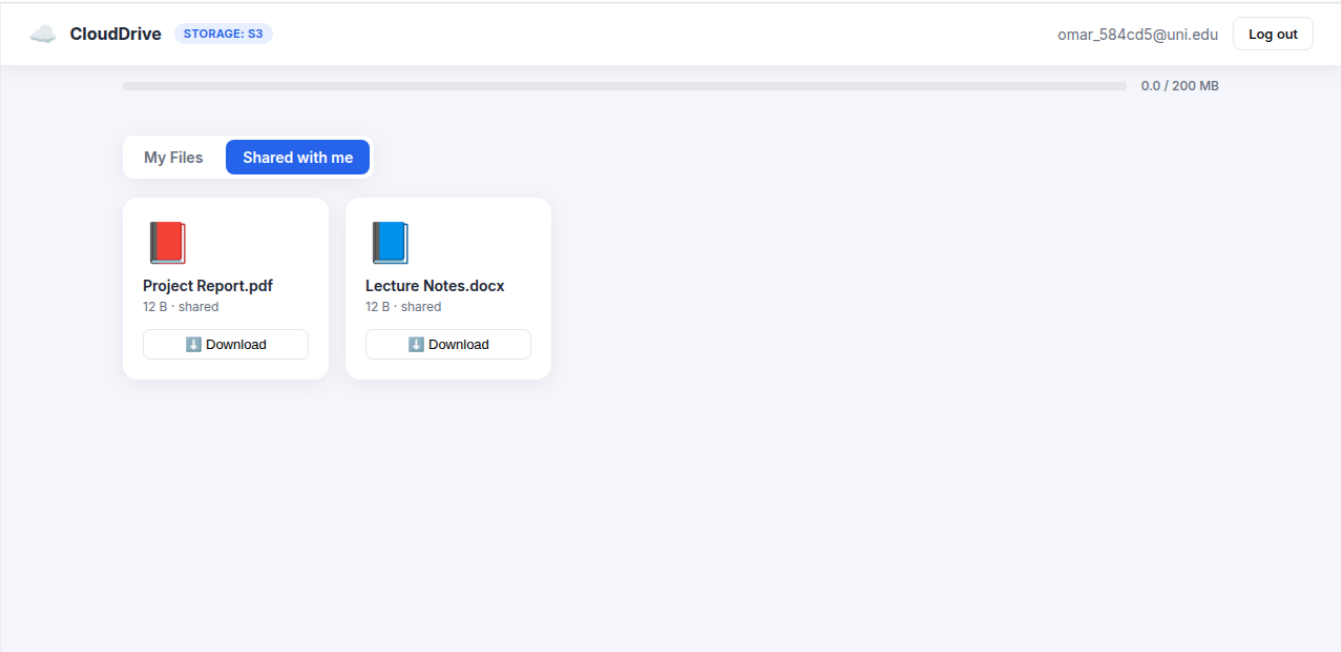


Figure 7: 'Shared with me' — files another user shared.

9. Cloud Deployment

Files are stored in a real Amazon S3 bucket using pre-signed URLs. The application can be run/deployed three ways, all sharing the same S3 storage:

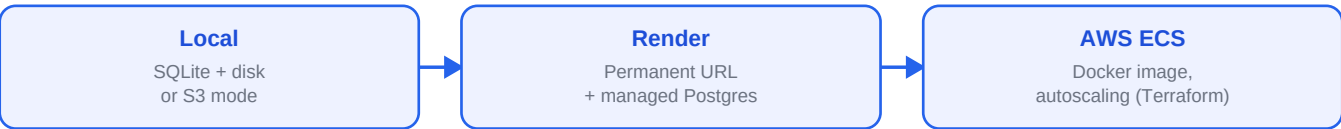


Figure 8: Deployment options (all use Amazon S3 for files).

For demonstrations, a Cloudflare tunnel exposes the local app on a public HTTPS URL so it can be opened from any device. The full AWS stack is defined in Terraform (validated) and deploys via the CI/CD pipeline below.

10. Testing

Nineteen automated tests (pytest) cover the critical paths and security:

Area	Examples	Tests
Authentication	signup, login, wrong password, duplicate email	6
Files	upload lifecycle, listing, quota, delete, isolation	7
Sharing & security	user share, public link, tampered/expired URL	6

All 19 tests pass and run automatically on every push via GitHub Actions.

11. CI/CD Pipeline

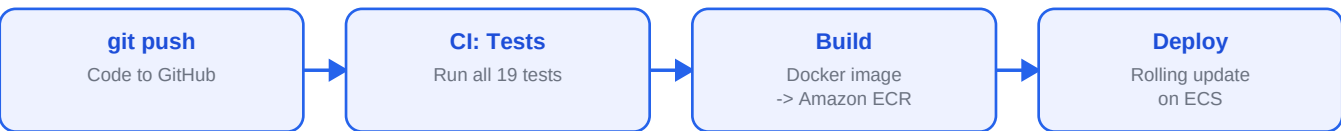


Figure 9: Continuous integration & deployment flow.

12. Cost & Free Tier

Service	Cost note
Amazon S3	Free tier: 5 GB + 20k GETs + 2k PUTs — demo cost ~ \$0
IAM / ECR	Free
RDS (db.t3.micro)	Free-tier eligible for 12 months
ALB / NAT / ECS	Billed hourly — destroyed after demos via 'terraform destroy'
Render / Tunnel	Free tiers

13. Conclusion & Future Work

CloudDrive delivers a complete, working, and tested cloud application that demonstrates the key principles of cloud computing — object storage, pre-signed URLs, managed databases, stateless scalable services, least-privilege security, infrastructure as code, and automated delivery. It runs on real AWS S3 and is fully reproducible from version control.

- Future: multipart upload for very large files (S3 multipart + Lambda confirm).
- Future: file previews and full-text search over metadata.
- Future: organisation/team workspaces and granular permissions.

Appendix — Run locally: `bash run.sh` · Run tests: `bash run_tests.sh` · Public demo: `bash demo.sh`